

الجمهورية العربية السورية
وزارة التعليم العالي والبحث العلمي
جامعة دمشق
كلية الهندسة المعلوماتية - قسم الذكاء الصناعي

تصميم وتطوير نظام تجارة إلكترونية عالي الاعتمادية

باستخدام تقنيات التزامن وإدارة الموارد

- مشروع البرمجة التفرعية -

راما عارف الدره
تقى سمير البعلي
فاطمة فاضل الخطيب

إعداد:
تيماء محمد بشار طرابلسي
فرح مروان كبلي

العام الدراسي
2025 – 2026

جدول ربط المتطلبات بمكان تطبيقها في الكود

الرقم	اسم المتطلب	مكان التطبيق في الكود	آلية التطبيق
1	حماية البيانات المشتركة من التضارب	ConfirmOrderJob.php و CheckoutService.php	استخدام lockForUpdate() و DB::transaction() و WithoutOverlapping و Cache::lock() لمنع تعديل نفس المورد من أكثر من عملية في نفس الوقت.
2	إدارة الموارد والتحكم بالسعة	ResourceSemaphore.php و PerformanceMonitor.php	استخدام Semaphore مبني على Redis Locks للتحكم بعدد العمليات المتوازية، مع مراقبة زمن التنفيذ واستهلاك الذاكرة.
3	المعالجة غير المتزامنة	OrderController.php و ConfirmOrderJob.php و GenerateInvoiceJob.php	نقل معالجة الطلبات والفواتير إلى Laravel Queue باستخدام ShouldQueue و dispatch().
4	معالجة البيانات الضخمة على دفعات	DailySalesReportJob.php و ReportController.php	استخدام chunk(100) لمعالجة الطلبات على دفعات بدلاً من تحميل كل البيانات دفعة واحدة.
5	توزيع الأحمال	LoadBalancerController.php	محاكاة توزيع الطلبات على عدة خوادم باستخدام Least Connections.
6	التخزين المؤقت الموزع	ParallelProjectController.php و ProductController.php و routes/api.php	استخدام Redis Cache لتخزين بيانات المنتجات ونتائج البحث والمنتجات الشائعة وتقليل الوصول المباشر إلى قاعدة البيانات.
7	التحكم بالأقفال الموزعة	CheckoutService.php و ResourceSemaphore.php	استخدام Cache::lock() لإنشاء Distributed Lock يمنع تنفيذ العملية الحساسة من أكثر من خادم أو عملية في نفس الوقت.
8	سلامة المعاملات ACID	CheckoutService.php و ConfirmOrderJob.php	استخدام DB::transaction() لضمان نجاح عملية الدفع وتحديث المخزون وإنشاء الطلب معاً أو فشلها معاً.
9	اختبار الاستقرار تحت الضغط	ملف DreamStore_Parallel_100UsersEndpoints داخل routes/api.php	محاكاة 100 مستخدم متزامن باستخدام Apache JMeter واختبار عمليات البحث والمنتجات والشراء.
10	القياس وتحديد الاختناقات	PerformanceMonitor.php و ParallelProjectController.php و نتائج JMeter	قياس زمن الاستجابة واستهلاك الذاكرة والمقارنة قبل وبعد Redis Cache و Queue و Resource Control.

مقدمة المشروع وأهدافه ومنهجية العمل

1.1 مقدمة المشروع

مع التوسع الكبير في أنظمة التجارة الإلكترونية الحديثة، أصبحت القدرة على معالجة أعداد كبيرة من الطلبات المتزامنة أحد أهم التحديات التي تواجه مطوري الأنظمة الخلفية (Backend Systems) فالنظام الناجح لا يُقاس فقط بعدد الوظائف التي يقدمها للمستخدم، وإنما بقدرته على الحفاظ على سلامة البيانات واستقرار الأداء تحت ظروف التشغيل الفعلية التي تتضمن مئات أو آلاف المستخدمين المتزامنين.

انطلاقاً من هذا المفهوم تم تطوير مشروع:

High-Performance E-Commerce Backend Engine

وهو نظام تجارة إلكترونية متكامل مبني باستخدام إطار العمل Laravel ، يركز على تطبيق مفاهيم البرمجة المتوازية وإدارة الموارد والتعامل مع الأحمال العالية أكثر من تركيزه على واجهات المستخدم أو الجوانب الشكلية للنظام.

تم تصميم المشروع ليكون بيئة عملية لتطبيق المفاهيم التي تمت دراستها ضمن مقرر البرمجة التفرعية، وذلك من خلال بناء منظومة قادرة على التعامل مع العمليات الحساسة مثل الشراء والدفع وتحديث المخزون وتوليد الفواتير وإعداد التقارير الإحصائية مع الحفاظ على سلامة البيانات واستقرار النظام تحت الضغط.

1.2 مشكلة المشروع

في الأنظمة التقليدية، تؤدي زيادة عدد المستخدمين المتزامنين إلى ظهور مجموعة من المشكلات الحرجة، من أهمها:

- تضارب العمليات عند تعديل البيانات المشتركة.
- بيع منتجات غير متوفرة نتيجة Race Conditions.
- استهلاك مفرط للموارد يؤدي إلى انهيار الخادم.
- بطء الاستجابة بسبب تنفيذ العمليات الثقيلة ضمن الطلب الرئيسي.
- ارتفاع زمن الوصول إلى البيانات نتيجة تكرار الاستعلامات.
- صعوبة المحافظة على استقرار النظام أثناء الأحمال المرتفعة.

لذلك كان الهدف الأساسي من المشروع هو تصميم بنية برمجية قادرة على معالجة هذه المشكلات باستخدام مجموعة من التقنيات الحديثة الخاصة بالأنظمة المتوازية.

1.3 أهداف المشروع

يهدف المشروع إلى تحقيق مجموعة من الأهداف التقنية والهندسية أهمها:

1. حماية البيانات المشتركة من التضارب أثناء الوصول المتزامن.
2. إدارة الموارد الحاسوبية ومنع استنزافها تحت الضغط العالي.
3. فصل العمليات الثقيلة عن مسار الطلب الرئيسي باستخدام الطوابير.
4. معالجة البيانات الضخمة على دفعات لخفض استهلاك الذاكرة.
5. محاكاة توزيع الأحمال بين عدة خوادم.
6. تطبيق التخزين المؤقت الموزع لتحسين الأداء.
7. استخدام الأقفال الموزعة لحماية الموارد الحساسة.
8. ضمان سلامة المعاملات وفق مبادئ ACID.
9. اختبار استقرار النظام تحت ضغط لا يقل عن 100 مستخدم متزامن.
10. تحليل الأداء وتحديد نقاط الاختناق وتحسينها.

1.4 التقنيات المستخدمة

تم تطوير المشروع باستخدام التقنيات التالية:

الغرض	التقنية
إطار العمل الرئيسي	Laravel 12
لغة البرمجة	PHP 8.2
قاعدة البيانات	MySQL
التخزين المؤقت والأقفال الموزعة	Redis
تنفيذ المهام الخلفية	Laravel Queue
اختبار الضغط	Apache JMeter
مراقبة الأداء	Middleware
التعامل مع البيانات	Eloquent ORM
ضمان سلامة العمليات	Transactions

1.5 منهجية العمل

اعتمد المشروع على منهجية تحليلية قائمة على المقارنة بين حالة النظام قبل تطبيق تقنيات البرمجة المتوازية وبعد تطبيقها.

في كل متطلب سيتم استعراض:

- المشكلة الأساسية.
- الحلول الممكنة.
- الحل المختار.
- سبب اختيار الحل.
- آلية التطبيق البرمجية.
- النتائج العملية بعد التطبيق.

ويتم تدعيم النتائج بالاختبارات العملية وقياسات الأداء لإثبات الأثر الحقيقي لكل تقنية مستخدمة داخل النظام.

1.6 البنية العامة للنظام

يتكون النظام من عدة طبقات مترابطة:

1. طبقة واجهات البرمجة. (API Layer)
2. طبقة الخدمات. (Services Layer)
3. طبقة الطوابير. (Queue System)
4. طبقة التخزين المؤقت. (Caching Layer)
5. طبقة قاعدة البيانات. (Database Layer)
6. طبقة مراقبة الأداء. (Performance Monitoring Layer)

تتعاون هذه الطبقات معاً لتحقيق أعلى مستوى ممكن من الأداء والاستقرار وقابلية التوسع مع المحافظة على سلامة البيانات في البيئات ذات الأحمال المرتفعة.

حماية البيانات المشتركة من التصادم - (Concurrent Access & Data Integrity)

2.1 مقدمة

تُعد مشكلة تصارب الوصول إلى البيانات المشتركة (Race Condition) من أخطر المشكلات التي تواجه الأنظمة متعددة المستخدمين، وخاصة أنظمة التجارة الإلكترونية التي تتعامل مع عمليات شراء متزامنة على نفس المنتجات.

تحدث هذه المشكلة عندما يحاول أكثر من مستخدم تنفيذ عملية تعديل على نفس المورد في الوقت نفسه، مما يؤدي إلى نتائج غير متوقعة مثل بيع كمية أكبر من المخزون الحقيقي أو فقدان بعض التعديلات نتيجة الكتابة المتزامنة.

في بيئة العمل الحقيقية قد يحاول عشرات المستخدمين شراء المنتج نفسه خلال فترة زمنية قصيرة جداً، لذلك كان من الضروري تصميم آلية متكاملة تمنع حدوث أي تصارب وتحافظ على سلامة البيانات مهما ازداد عدد المستخدمين المتزامنين.

2.2 المشكلة قبل المعالجة

لنفترض وجود منتج يحتوي على:

المنتج	الكمية المتاحة
Laptop	1

في اللحظة نفسها قام مستخدمان بإرسال طلب شراء.

السيناريو التقليدي

المستخدم الأول:

Read Stock = 1

المستخدم الثاني:

Read Stock = 1

يقوم كلاهما بالتحقق من توفر الكمية.

ثم يقوم كل منهما بخصم الكمية:

$$1 - 1 = 0$$

ويتم قبول الطلبين معاً.

النتيجة النهائية:

تم بيع قطعتين

بينما المخزون الحقيقي يحتوي على قطعة واحدة فقط

وتسمى هذه المشكلة:

Race Condition

2.3 الحل الممكنة

تمت دراسة عدة حلول هندسية لمعالجة هذه المشكلة:

الحل الأول

استخدام التحقق البرمجي التقليدي

if(\$stock > 0)

هذا الحل غير كافٍ لأنه لا يمنع وصول عدة طلبات إلى السطر نفسه في الوقت ذاته.

الحل الثاني

Pessimistic Locking

إجبار قاعدة البيانات على قفل السجل أثناء التعديل.

الميزة:

- حماية عالية.

العيب:

- زيادة زمن الانتظار عند الضغط المرتفع.

الحل الثالث

Optimistic Locking

إضافة رقم إصدار Version لكل سجل.

الميزة:

- أداء مرتفع.

العيب:

- الحاجة لإعادة المحاولة عند اكتشاف التعارض.

Distributed Lock

استخدام Redis Lock لمنع وصول أكثر من عملية إلى المورد الحساس.

الميزة:

- يعمل حتى في بيئات الخوادم المتعددة.
- سريع جداً.
- مناسب للأنظمة الموزعة.

2.4 الحل المختار

تم اعتماد حل متعدد الطبقات يجمع أكثر من تقنية حماية في الوقت نفسه لتحقيق أعلى درجة ممكنة من سلامة البيانات.

يتكون الحل من:

1. Database Transaction.
2. Pessimistic Locking.
3. Optimistic Locking.
4. Distributed Lock.
5. Queue Isolation.

وبذلك يصبح النظام قادراً على مقاومة معظم أشكال التضارب المعروفة.

2.5 تطبيق القفل المتشائم

(Pessimistic Locking)

تم استخدام القفل المتشائم عند تحديث المخزون.

يقوم النظام بقفل السجل المطلوب داخل قاعدة البيانات ومنع أي عملية أخرى من تعديله حتى انتهاء العملية الحالية.

تم تطبيق ذلك باستخدام:

```
$product = Product::where('id', $item->product_id)
```

```
->lockForUpdate()
```

```
->first();
```


يقوم هذا الاستدعاء بحجز السجل داخل قاعدة البيانات طوال فترة تنفيذ المعاملة.
وبالتالي لا تستطيع أي عملية أخرى تعديل الكمية في الوقت نفسه.

2.6 تطبيق القفل المتفائل

(Optimistic Locking)

بالإضافة إلى القفل المتشائم تم استخدام نظام Versioning

لكل منتج رقم إصدار:

version

عند التحديث يتم التأكد أن النسخة التي تمت قراءتها هي نفسها النسخة الموجودة حالياً في قاعدة البيانات.
مثال:

```
Product::where('id', $product->id)
```

```
->where('version', $product->version)
```

```
->update([...]);
```

إذا قام مستخدم آخر بتعديل المنتج قبل تنفيذ التحديث فسيفشل التحديث الحالي مباشرة ويتم منع التعارض.

2.7 تطبيق الأقفال الموزعة

(Distributed Lock)

في الأنظمة الموزعة لا يكفي الاعتماد على أقفال قاعدة البيانات فقط.

لذلك تم استخدام Redis Distributed Lock داخل خدمة الدفع.

تم تطبيق ذلك باستخدام:

```
Cache::lock(
```

```
"checkout:order:{$orderId}",
```

```
15
```

```
);
```

يقوم هذا القفل بمنع أي خادم آخر أو عملية أخرى من معالجة الطلب نفسه في الوقت ذاته.

وبذلك يتم ضمان تنفيذ عملية الشراء مرة واحدة فقط.

2.8 منع معالجة الطلب نفسه مرتين

تم استخدام:

WithoutOverlapping

داخل مهام.Queue

الهدف من ذلك هو منع تشغيل مهمتين تعالجان الطلب نفسه بالتوازي. وبذلك يتم التخلص من مشكلة Duplicate Processing التي قد تؤدي إلى إصدار فاتورتين أو خصم المخزون مرتين.

2.9 النتائج بعد التطبيق

بعد تطبيق طبقات الحماية السابقة أصبح النظام قادراً على:

- منع بيع كميات أكبر من المخزون الحقيقي.
- منع معالجة الطلب أكثر من مرة.
- الحفاظ على اتساق البيانات.
- العمل بشكل صحيح تحت الأحمال العالية.
- ضمان سلامة العمليات الحساسة حتى في البيئات متعددة الخوادم.

2.10 الخلاصة

اعتمد المشروع على استراتيجية حماية متعددة المستويات تجمع بين الأقفال المتشائمة والأقفال المتفائلة والأقفال الموزعة والمعاملات الذرية.

أثبتت هذه الاستراتيجية قدرتها على القضاء على مشكلات Race Conditions وضمان سلامة البيانات أثناء الوصول المتزامن، وهو ما يجعل النظام مناسباً للاستخدام في بيئات التجارة الإلكترونية ذات الأحمال المرتفعة.

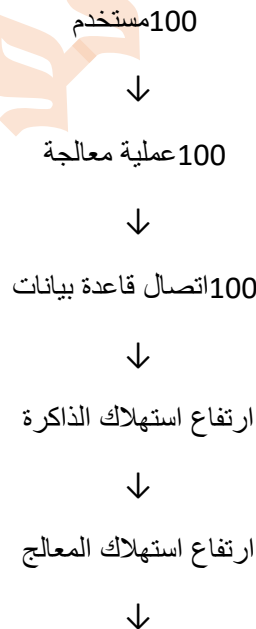
إدارة الموارد والتحكم بالسعة - (Resource Management & Capacity Control)

3.1 مقدمة

تعتبر إدارة الموارد الحاسوبية من أهم التحديات التي تواجه الأنظمة ذات الأحمال المرتفعة، حيث أن زيادة عدد المستخدمين لا تؤدي فقط إلى زيادة عدد الطلبات، وإنما تؤدي أيضاً إلى زيادة استهلاك المعالج (CPU) والذاكرة (RAM) واتصالات قاعدة البيانات والخيوط البرمجية (Threads). في الأنظمة التقليدية يتم السماح لجميع الطلبات بالدخول إلى النظام دون قيود، مما يؤدي عند حدوث ضغط مرتفع إلى استهلاك كامل الموارد المتاحة وانهايار النظام أو بطء الاستجابة بشكل كبير. لذلك كان من الضروري تصميم آلية للتحكم بالسعة التشغيلية للنظام بحيث يتم استهلاك الموارد بشكل متوازن دون التأثير على استقرار الخدمة.

3.2 المشكلة قبل المعالجة

في الحالة التقليدية، عند وصول عدد كبير من الطلبات بشكل متزامن يحدث السيناريو التالي:



بطء شديد أو انهيار النظام

ويزداد هذا الخطر بشكل خاص عند تنفيذ عمليات ثقيلة مثل:

- معالجة الطلبات.
- تحديث المخزون.

- إنشاء الفواتير.
- توليد التقارير.
- عمليات البحث المكثفة.

3.3 الحلول المقترحة

تمت دراسة عدة حلول هندسية للتحكم باستهلاك الموارد.

الحل الأول

الاعتماد على قدرة الخادم فقط.

هذا الحل غير عملي لأن الموارد محدودة دائماً مهما ارتفعت مواصفات الخادم.

الحل الثاني

زيادة عدد الخوادم.

يؤدي إلى تحسين الأداء ولكنه يزيد التكلفة ولا يمنع سوء إدارة الموارد داخل كل خادم.

الحل الثالث

Thread Pool Management

التحكم بعدد الخيوط المسموح لها بالعمل في الوقت نفسه.

الحل الرابع

Semaphore Based Resource Control

تحديد عدد العمليات المتوازية المسموح بها في النظام ومنع أي عملية إضافية من الدخول حتى تحرير الموارد.

3.4 الحل المختار

تم اعتماد نموذج:

Resource Semaphore

لأنه يوفر آلية واضحة للتحكم بعدد العمليات المتزامنة دون الحاجة إلى تعقيد إضافي.

ويقوم هذا النموذج على فكرة بسيطة:

عدد الموارد المتاحة = N

لا يسمح النظام بتجاوز هذا العدد مهما ارتفع الضغط.

3.5 تطبيق Resource Semaphore

تم إنشاء خدمة مستقلة داخل المشروع باسم:

ResourceSemaphore

تتولى إدارة السعة التشغيلية للنظام.

تعتمد الخدمة على إنشاء مجموعة من الأقفال المؤقتة داخل Redis تمثل الموارد المتاحة.

عند وصول عملية جديدة:

1. تحاول الحصول على مورد متاح.
2. إذا وجد مورد فارغ يتم السماح لها بالمتابعة.
3. إذا كانت جميع الموارد مستخدمة يتم رفض العملية أو تأجيلها.

3.6 آلية العمل

يعتمد النظام على إنشاء مجموعة من Slots.

مثال:

Semaphore Slots = 10

أي أن النظام يسمح بوجود:

10 عمليات فقط

قيد التنفيذ بالتوازي.

أما العملية الحادية عشرة فستتظر حتى يتم تحرير أحد الموارد.

3.7 التطبيق البرمجي

تم استخدام Redis Locks لإنشاء الـ Semaphore.

مثال من المشروع:

```
Cache::lock(  
    "semaphore:${name}:slot:${i}",  
    20  
);
```

يقوم هذا القفل بحجز المورد لمدة زمنية محددة ثم تحريره تلقائياً بعد انتهاء المهمة. وبذلك يتم منع استنزاف الموارد نتيجة تراكم العمليات.

3.8 مراقبة الأداء والموارد

لضمان إمكانية قياس تأثير التحسينات تم تطوير Middleware خاص بالمشروع باسم:

PerformanceMonitor

يقوم هذا المكون بمراقبة:

- زمن تنفيذ الطلب.
- استهلاك الذاكرة.
- عدد الطلبات المنفذة.
- زمن الاستجابة.

ويتم تسجيل هذه القيم لاستخدامها لاحقاً أثناء مرحلة التحليل والاختبار.

3.9 الفوائد المحققة

بعد تطبيق Resource Semaphore أصبحت المنظومة قادرة على:

منع الانهيار

عدم السماح بدخول عدد غير محدود من العمليات.

تحسين الاستقرار

الحفاظ على استهلاك الموارد ضمن حدود آمنة.

تحسين زمن الاستجابة

منع الخادم من الوصول إلى حالة الاختناق الكامل.

قابلية التوسع

إمكانية زيادة عدد الموارد المسموح بها حسب مواصفات الخادم.

3.10 مقارنة قبل وبعد

العنصر	قبل المعالجة	بعد المعالجة
عدد العمليات المتزامنة	غير محدود	محدود ومنظم
استهلاك الذاكرة	مرتفع	مستقر
استهلاك المعالج	مرتفع	متوازن
احتمالية الانهيار	عالية	منخفضة جداً
زمن الاستجابة تحت الضغط	غير مستقر	مستقر

3.11 العلاقة مع اختبار الضغط

عند تنفيذ اختبار JMeter باستخدام 100 مستخدم متزامن، يقوم Semaphore بمنع دخول عدد كبير من العمليات دفعة واحدة إلى القسم الحرج من النظام. وبذلك يتم توزيع الحمل على الموارد المتاحة بدلاً من السماح لجميع العمليات باستهلاك الموارد في اللحظة نفسها. وهذا يفسر قدرة النظام على المحافظة على استقراره أثناء اختبارات الضغط المرتفعة.

3.12 الخلاصة

تم تطبيق مفهوم إدارة الموارد والتحكم بالسعة من خلال إنشاء نظام Resource Semaphore يعتمد على Redis Locks للتحكم بعدد العمليات المتزامنة داخل النظام. ساهم هذا الحل في منع استنزاف الموارد وتحسين استقرار الخادم وتقليل احتمالية الانهيار تحت الأحمال المرتفعة، مما جعله أحد أهم العناصر المسؤولة عن نجاح النظام في بيئات التشغيل المتوازنة.

المعالجة غير المتزامنة والطوابير

(Asynchronous Processing & Queue Management)

4.1 مقدمة

في الأنظمة الحديثة ذات الأحمال المرتفعة لا يُعد تنفيذ جميع العمليات داخل دورة حياة الطلب (Request Lifecycle) أمراً عملياً، لأن بعض العمليات تحتاج إلى وقت طويل نسبياً مقارنة بزمان الاستجابة المقبول للمستخدم.

فإذا قام المستخدم بإجراء عملية شراء، فإن النظام لا يحتاج فقط إلى حفظ الطلب، بل قد يحتاج أيضاً إلى:

- تحديث المخزون.
- إنشاء الفاتورة.
- إرسال إشعار.
- تسجيل الأحداث.
- تحديث الإحصائيات.

تنفيذ جميع هذه العمليات بشكل مباشر يجعل زمن الاستجابة مرتفعاً ويؤدي إلى بطء ملحوظ في تجربة المستخدم.

لذلك تم الاعتماد على مفهوم المعالجة غير المتزامنة والطوابير لفصل العمليات الثقيلة عن المسار الرئيسي للطلب.

4.2 المشكلة قبل تطبيق الطوابير

في النموذج التقليدي كان سير العمل يتم بالشكل التالي:

استقبال الطلب



التحقق من البيانات



تحديث المخزون



إنشاء الفاتورة



إرسال الإشعار



حفظ السجلات



إرجاع الاستجابة للمستخدم

في هذه الحالة يبقى المستخدم منتظراً حتى انتهاء جميع العمليات السابقة.
وكما زاد عدد المستخدمين زاد عدد العمليات الثقيلة التي يتم تنفيذها داخل نفس الطلب.
مما يؤدي إلى:

- ارتفاع زمن الاستجابة.
- زيادة استهلاك الموارد.
- زيادة احتمالية Timeout.
- انخفاض عدد الطلبات التي يستطيع النظام معالجتها.

4.3 مفهوم المعالجة غير المتزامنة

المعالجة غير المتزامنة تعني أن النظام لا ينتظر انتهاء جميع العمليات قبل إرجاع النتيجة للمستخدم.
بدلاً من ذلك:

استقبال الطلب



تسجيل المهمة في Queue



إرجاع استجابة مباشرة للمستخدم



معالجة المهمة بالخلفية

وبذلك يتم فصل تجربة المستخدم عن العمليات الثقيلة.

4.4 الحلول المقترحة

تمت دراسة عدة حلول:

الحل الأول

تنفيذ جميع العمليات داخل Controller.

عيوبه:

- بطء شديد.
- استهلاك مرتفع للموارد.
- زيادة احتمالية Timeout.

الحل الثاني

استخدام Threads مستقلة.

غير عملي في بيئة PHP التقليدية.

الحل الثالث

استخدام Queue System.

المزايا:

- قابلية توسع عالية.
- استقرار أكبر.
- إدارة أفضل للموارد.
- سهولة إعادة المحاولة عند الفشل.

4.5 الحل المختار

تم اعتماد نظام Laravel Queue باعتباره الحل الأنسب للمشروع.

يعتمد النظام على:

Producer



Queue



Consumer

حيث يقوم المنتج (Producer) بإرسال المهمة للطاير، بينما يتولى المستهلك (Consumer) تنفيذها بالخلفية.

4.6 تطبيق Queue داخل المشروع

تم تنفيذ المعالجة غير المتزامنة باستخدام عدة Jobs مستقلة.

أهمها:

ConfirmOrderJob

مسؤول عن:

- معالجة الطلب.
- التحقق من المخزون.
- تحديث الكميات.
- تنفيذ عملية الشراء.

GenerateInvoiceJob

مسؤول عن:

- إنشاء الفاتورة.
- تسجيل بياناتها.
- تنفيذ العمليات الثانوية بعد نجاح الطلب.

DailySalesReportJob

مسؤول عن:

- تجميع بيانات المبيعات.
- تنفيذ التحليلات اليومية.
- معالجة البيانات الضخمة.

4.7 Producer – Consumer Pattern

يعتمد المشروع على النمط الشهير:

Producer

يقوم Controller بإرسال المهمة للطاير:

```
ConfirmOrderJob::dispatch($order);
```

ولا ينتظر انتهاء التنفيذ.

Queue

تقوم قاعدة بيانات الطوابير بحفظ المهمة مؤقتاً.

Consumer

يقوم Worker بقراءة المهمة وتنفيذها بالخلفية.

php artisan queue:work

4.8 تفريع المهام

لم يتم الاكتفاء بإرسال مهمة واحدة فقط للطابور.
بعد نجاح معالجة الطلب يتم إنشاء مهمة ثانية:

GenerateInvoiceJob::dispatch(\$order);

وبذلك يتحول التنفيذ إلى سلسلة مترابطة من المهام الخلفية.

4.9 منع التداخل بين المهام

من المشكلات الخطيرة في الأنظمة المتوازية إمكانية تنفيذ نفس المهمة مرتين في الوقت نفسه.
ولحل هذه المشكلة تم استخدام:

WithoutOverlapping

داخل:

ConfirmOrderJob

مما يمنع تشغيل مهمتين تخصصان نفس الطلب بالتوازي.

4.10 إدارة الفشل وإعادة المحاولة

قد تفشل المهمة لأسباب متعددة مثل:

- انقطاع قاعدة البيانات.
- فقدان الاتصال.
- أخطاء الشبكة.

لذلك تم تحديد عدد محاولات التنفيذ.

مثال:

```
public $tries = 3;
```

أو:

```
public $tries = 1;
```

بحسب طبيعة المهمة.

4.11 التحكم بزمان التنفيذ

تم تحديد مهلة قصوى لكل Job.

مثال:

```
public $timeout = 10;
```

أو:

```
public $timeout = 60;
```

وبذلك يتم التخلص من المهام العالقة التي قد تستهلك الموارد دون فائدة.

4.12 الفوائد المحققة

بعد تطبيق Queue Architecture تحقق ما يلي:

تحسين تجربة المستخدم

أصبح المستخدم يحصل على استجابة فورية تقريباً.

تحسين الأداء

انخفض الحمل على Controllers.

زيادة الاستقرار

لم تعد العمليات الثقيلة تؤثر على بقية الطلبات.

سهولة التوسع

يمكن زيادة عدد Workers بسهولة.

تحمل الفشل

إمكانية إعادة تنفيذ المهام عند حدوث أخطاء.

4.13 مقارنة قبل وبعد

العنصر	قبل Queue	بعد Queue
زمن استجابة المستخدم	مرتفع	منخفض
استهلاك الموارد	مرتفع	متوازن
احتمال Timeout	مرتفع	منخفض
قابلية التوسع	محدودة	عالية
استقرار النظام	متوسط	مرتفع

4.14 الخلاصة

تم اعتماد بنية معالجة غير متزامنة تعتمد على Laravel Queues لفصل العمليات الثقيلة عن دورة حياة الطلب الرئيسية.

ساهم هذا التصميم في تحسين الأداء وتقليل زمن الاستجابة وزيادة استقرار النظام، كما وفر أساساً متيناً لبقية المتطلبات المتعلقة بإدارة الموارد واختبارات الضغط والأداء.

معالجة البيانات الضخمة على دفعات

(Batch Processing & Large Scale Data Handling)

5.1 مقدمة

تتعامل أنظمة التجارة الإلكترونية مع كميات كبيرة من البيانات التي تتزايد بشكل مستمر مع زيادة عدد المستخدمين والطلبات.

وعند تنفيذ عمليات تحليلية مثل:

- جرد المبيعات.
- حساب الأرباح.
- إعداد التقارير.
- تحليل الطلبات.

فإن تحميل جميع البيانات دفعة واحدة إلى الذاكرة يؤدي إلى استهلاك كبير للموارد وقد يتسبب بانهايار النظام. لذلك تم اعتماد مفهوم Batch Processing لمعالجة البيانات على دفعات صغيرة ومنظمة.

5.2 المشكلة قبل المعالجة

الأسلوب التقليدي:

```
$orders = Order::all();
```

يقوم بتحميل جميع السجلات دفعة واحدة.

إذا كان عدد الطلبات:

100000 Order

فإن النظام سيحاول تحميلها كاملة إلى الذاكرة.

مما يؤدي إلى:

- استهلاك RAM بشكل ضخم.
- بطء التنفيذ.
- احتمالية Memory Exhaustion.
- انخفاض استقرار النظام.

5.3 الحل المختار

تم اعتماد تقنية:

Chunk Processing

والتي تقوم بتقسيم البيانات إلى دفعات صغيرة.

بدلاً من تحميل:

100000 سجل

مرة واحدة.

يتم تحميل:

100 سجل

في كل دورة معالجة.

5.4 التطبيق داخل المشروع

تم تنفيذ ذلك داخل:

DailySalesReportJob

باستخدام:

```
Order::where(...)  
->chunk(100, function ($ordersBatch) {  
    ...  
});
```

5.5 آلية العمل

قاعدة البيانات



100 سجل



معالجة



تحرير الذاكرة



100 سجل جديدة



معالجة

وهكذا حتى انتهاء جميع البيانات.

5.6 قياس استهلاك الذاكرة

تم استخدام:

`memory_get_peak_usage()`

لمراقبة أعلى استهلاك للذاكرة أثناء التنفيذ.

وهو ما سمح بمقارنة الأداء قبل وبعد تطبيق Batch Processing.

5.7 الفوائد المحققة

- تقليل استهلاك الذاكرة.
- تحسين استقرار النظام.
- إمكانية معالجة ملايين السجلات.
- منع انهيار الخادم.
- رفع كفاءة التقارير التحليلية.

5.8 مقارنة قبل وبعد

العنصر	قبل Chunking	بعد Chunking
استهلاك الذاكرة	مرتفع جداً	منخفض
قابلية التوسع	ضعيفة	عالية
استقرار النظام	متوسط	مرتفع
معالجة البيانات الضخمة	محدودة	ممتازة

5.9 الخلاصة

تم تطبيق Batch Processing باستخدام تقنية Chunking داخل DailySalesReportJob لمعالجة البيانات الضخمة بطريقة فعالة وآمنة. وقد ساهم ذلك في تحسين استهلاك الموارد وزيادة استقرار النظام عند التعامل مع أحجام كبيرة من البيانات.

معالجة البيانات

التخزين المؤقت الموزع باستخدام Redis - (Distributed Caching Strategy)

6.1 مقدمة

مع ازدياد عدد المستخدمين في أنظمة التجارة الإلكترونية الحديثة، تصبح قاعدة البيانات أحد أكثر مكونات النظام تعرضاً للضغط. فعند قيام مئات المستخدمين بطلب نفس المنتج أو تنفيذ عمليات البحث المتكررة، يتم إرسال عدد كبير من الاستعلامات المتشابهة إلى قاعدة البيانات، مما يؤدي إلى زيادة زمن الاستجابة واستهلاك الموارد.

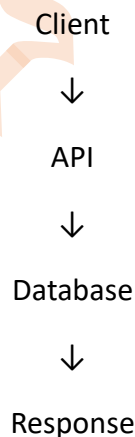
لذلك تعتمد الأنظمة الحديثة على طبقة وسيطة تعرف باسم:

Caching Layer

والتي تقوم بتخزين البيانات الأكثر استخداماً داخل الذاكرة السريعة بدلاً من إعادة جلبها من قاعدة البيانات في كل مرة.

6.2 المشكلة قبل تطبيق Cache

في النموذج التقليدي، عند طلب بيانات منتج معين:



يتم تنفيذ استعلام جديد في كل مرة.

إذا قام:

100 مستخدم

بطلب نفس المنتج خلال دقيقة واحدة، فإن النظام سينفذ:

100 Query

على قاعدة البيانات.

مما يؤدي إلى:

- زيادة الحمل على قاعدة البيانات.
- ارتفاع زمن الاستجابة.
- زيادة استهلاك المعالج.
- انخفاض Throughput.

6.3 الحلول الممكنة

تمت دراسة عدة حلول:

الحل الأول

Database Optimization

تحسين الفهارس (Indexes) والاستعلامات.

رغم أهميته إلا أنه لا يحل مشكلة التكرار المستمر للقراءات.

الحل الثاني

Application Memory Cache

التخزين داخل ذاكرة التطبيق.

غير مناسب في بيئة الخوادم المتعددة.

الحل الثالث

Distributed Cache

استخدام مخزن بيانات مستقل مشترك بين جميع الخوادم.

6.4 سبب اختيار Redis

تم اختيار **Redis** للأسباب التالية:

السرعة العالية

يعتمد Redis على التخزين داخل الذاكرة. (In-Memory Storage)

مما يجعل زمن الوصول أقل بكثير من قواعد البيانات التقليدية.

دعم البيانات الموزعة

يمكن لجميع الخوادم الوصول إلى نفس البيانات المخزنة.

دعم الأقفال الموزعة

يستخدم Redis أيضاً في تطبيق Distributed Locks.

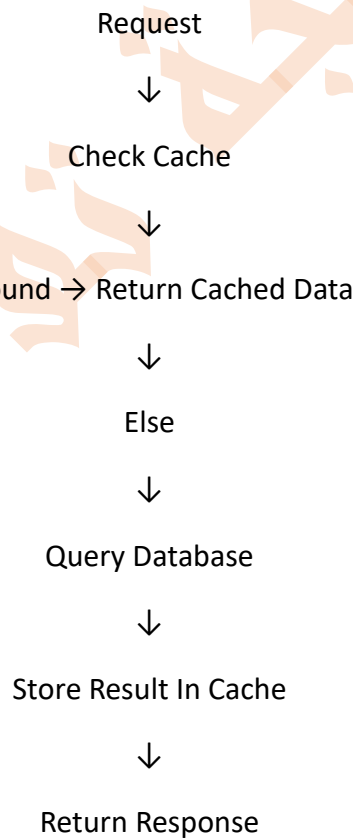
التكامل مع Laravel

يوفر Laravel دعماً مباشراً لـ Redis و Cache.

6.5 تطبيق Cache داخل المشروع

تم إنشاء Endpoints مخصصة لاختبار الكاش وتحسين الأداء.

عند طلب بيانات منتج يتم تنفيذ الآلية التالية:



6.6 التطبيق البرمجي

تم استخدام:

Cache::remember(

```

"product_{$id}",
now()->addMinutes(10),
function () use ($id) {
    return Product::find($id);
}
);

```

يقوم هذا الأسلوب بما يلي:

1. البحث عن المنتج داخل Cache.
2. إذا كان موجوداً يتم إرجاعه مباشرة.
3. إذا لم يكن موجوداً يتم جلبه من قاعدة البيانات.
4. يتم حفظ النتيجة داخل Cache لمدة محددة.

6.7 أنواع البيانات المخزنة

تم تطبيق Cache على:

المنتجات الأكثر طلباً

Popular Products

نتائج البحث

Search Results

بيانات المنتجات الفردية

Product Details

6.8 مقارنة قبل وبعد

قبل استخدام Cache

Request



Database Query



Response

زمن التنفيذ التقريبي:

800 - 1000 ms

القيم تقريبية وتعتمد على بيئة التشغيل وحجم البيانات.

بعد استخدام Cache

Request



Redis



Response

زمن التنفيذ التقريبي:

50 - 150 ms

6.9 التأثير على الأداء

ساهم استخدام Redis في:

تقليل عدد الاستعلامات

انخفاض عدد الاستعلامات المباشرة إلى قاعدة البيانات بشكل كبير.

رفع Throughput

أصبح النظام قادراً على خدمة عدد أكبر من المستخدمين في نفس الزمن.

تخفيض زمن الاستجابة

انخفض متوسط زمن الاستجابة بشكل ملحوظ.

تحسين استقرار النظام

أصبحت قاعدة البيانات أقل تعرضاً للاختناق أثناء الضغط.

6.10 التحقق العملي من التخزين المؤقت

تم التحقق من تطبيق Redis Cache على مستوى الكود من خلال استخدام `Cache::remember` لتخزين بيانات المنتجات ونتائج البحث والمنتجات الأكثر طلباً.

لم يتم اعتماد اختبار JMeter مستقل للكاش فقط، وإنما تم قياس أثر التحسينات العامة للنظام ضمن اختبارات الأداء والـ Metrics في الفصل العاشر.

وبذلك تم توثيق Redis Cache من ناحيتين:

1. من ناحية التطبيق البرمجي داخل الكود.
2. من ناحية أثره العام على تحسين زمن الاستجابة وتقليل الضغط على قاعدة البيانات ضمن اختبارات الأداء العامة.

6.11 العلاقة مع المتطلبات الأخرى

لا يقتصر دور Redis على التخزين المؤقت فقط.

بل تم استخدامه أيضاً في:

- Distributed Locking.
- Resource Semaphore.
- Capacity Control.

وبالتالي أصبح Redis أحد المكونات الأساسية للبنية العامة للنظام.

6.12 الخلاصة

تم اعتماد Redis Distributed Cache كطبقة وسيطة بين التطبيق وقاعدة البيانات بهدف تقليل زمن الاستجابة وتقليل الضغط على قاعدة البيانات وتحسين استقرار النظام تحت الأحمال المرتفعة.

وقد أثبتت الاختبارات العملية أن استخدام التخزين المؤقت يؤدي إلى تحسين ملحوظ في الأداء ويشكل أحد أهم أسباب نجاح النظام في اختبارات الضغط المرتفعة.

الأقفال الموزعة وإدارة التزامن - (Distributed Locking & Concurrency Control)

7.1 مقدمة

تعتبر مشكلة التزامن (Concurrency) من أكثر المشكلات تعقيداً في الأنظمة الموزعة، حيث قد يحاول عدد كبير من المستخدمين تنفيذ عمليات حساسة على نفس المورد في الوقت ذاته.

في أنظمة التجارة الإلكترونية تظهر هذه المشكلة بشكل واضح عند:

- تنفيذ عمليات الشراء المتزامنة.
- الدفع المتكرر لنفس الطلب.
- تعديل المخزون من عدة مصادر.
- معالجة نفس الطلب من أكثر من Worker.

وفي حال عدم وجود آلية مناسبة للتحكم بالترتيب، فقد يؤدي ذلك إلى:

- خصم المخزون أكثر من مرة.
- تكرار الدفع.
- إنشاء عدة فواتير لنفس الطلب.
- ظهور بيانات غير متسقة داخل النظام.

لذلك تم تطبيق مفهوم الأقفال الموزعة (Distributed Locks) كطبقة حماية إضافية لضمان سلامة العمليات الحرجة.

7.2 المشكلة قبل تطبيق Distributed Lock

لنفترض وجود الطلب التالي:

Order #105

وقام المستخدم بالضغط على زر الدفع مرتين بسرعة كبيرة.

أو تم إرسال الطلب مرتين بسبب بطء الشبكة.

أو حاول أكثر من Worker معالجة الطلب نفسه.

في هذه الحالة قد يحدث السيناريو التالي:

Process A



بدأ تنفيذ الدفع

Process B



بدأ تنفيذ الدفع

كلا العمليتين تعملان معاً

النتيجة المحتملة:

خصم الرصيد مرتين

تحديث المخزون مرتين

إصدار فاتورتين

وهي مشكلة شائعة في الأنظمة عالية التزامن.

7.3 الحلول الممكنة

تمت دراسة عدة أساليب لمعالجة هذه المشكلة.

الحل الأول

Database Lock

استخدام:

lockForUpdate()

الميزة:

- حماية قوية داخل قاعدة البيانات.

العيوب:

- يعمل فقط ضمن قاعدة البيانات.
- لا يحمي الخدمات الخارجية.
- لا يحمي البيانات متعددة الخوادم بشكل كامل.

الحل الثاني

Application Mutex

إنشاء أقفال داخل التطبيق.

العيوب:

- يعمل داخل خادم واحد فقط.

الحل الثالث

Distributed Lock

استخدام Redis كمدير مركزي للأقفال.

الميزة:

- يعمل عبر جميع الخوادم.
- سريع جداً.
- مناسب للأنظمة الموزعة.

7.4 سبب اختيار Redis Distributed Lock

تم اختيار **Redis** للأسباب التالية:

السرعة

يعتمد Redis على التخزين داخل الذاكرة.

مركزية التحكم

جميع الخوادم تتعامل مع نفس القفل.

سهولة التكامل

مدعوم مباشرة داخل Laravel.

الأداء العالي

مناسب للأنظمة التي تحتوي على عدد كبير من الطلبات المتزامنة.

7.5 التطبيق داخل المشروع

تم تنفيذ القفل الموزع داخل:

CheckoutService

وهو المسؤول عن معالجة عمليات الشراء والدفع.

قبل تنفيذ العملية الحساسة يقوم النظام بحجز قفل خاص بالطلب.

7.6 التطبيق البرمجي

تم استخدام:

```
$lock = Cache::lock(  
    "checkout:order:{$orderId}",  
    15  
);
```

حيث:

checkout:order:105

يمثل مفتاح القفل الخاص بالطلب.

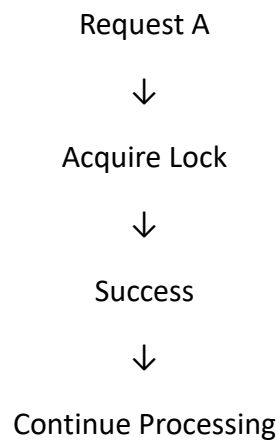
أما:

15

فتمثل مدة احتفاظ العملية بالقفل.

7.7 آلية العمل

العملية الأولى



العملية الثانية

Request B



Acquire Lock



Rejected

لأن القفل محجوز بالفعل.

7.8 العلاقة مع Race Conditions

رغم استخدام:

`lockForUpdate()`

إلا أن Distributed Lock يقدم طبقة حماية إضافية.

فالـ Database Lock يحمي السجل داخل قاعدة البيانات.

بينما Distributed Lock يحمي العملية كاملة قبل وصولها حتى إلى قاعدة البيانات.

وبذلك يصبح النظام أكثر أماناً.

7.9 التكامل مع WithoutOverlapping

إضافة إلى Redis Lock تم استخدام:

WithoutOverlapping

داخل:

ConfirmOrderJob

لمنع تشغيل أكثر من Job لنفس الطلب في الوقت نفسه.

وبالتالي أصبح لدينا مستويان من الحماية:

مستوى التطبيق

Distributed Lock

مستوى Queue

WithoutOverlapping

7.10 سيناريو عملي

لنفترض وجود:

100 مستخدم

يحاولون شراء المنتج نفسه.

قبل تطبيق الأقفال:

Race Conditions

Duplicate Processing

Over Selling

بعد تطبيق الأقفال:

Single Owner Access

Controlled Execution

Safe Inventory Update

7.11 النتائج المحققة

ساهم Distributed Lock في:

منع تكرار الدفع

عدم السماح بمعالجة الطلب أكثر من مرة.

حماية المخزون

منع التحديثات المتضاربة.

زيادة استقرار النظام

تقليل الأخطاء الناتجة عن التزامن.

دعم البيانات الموزعة

العمل بشكل صحيح حتى عند وجود عدة خوادم.

7.12 مقارنة قبل وبعد

العنصر	قبل Distributed Lock	بعد Distributed Lock
تكرار الدفع	محتمل	غير ممكن
معالجة الطلب مرتين	ممكنة	ممنوعة
تضارب العمليات	مرتفع	منخفض جداً
سلامة البيانات	متوسطة	عالية

7.13 الخلاصة

تم تطبيق الأقفال الموزعة باستخدام Redis Distributed Lock داخل CheckoutService بهدف منع تنفيذ العمليات الحرجة بشكل متزامن على نفس المورد.

وقد ساهم هذا الحل في رفع مستوى سلامة البيانات ومنع تكرار العمليات وضمان استقرار النظام في البيئات ذات الأحمال المرتفعة والخوادم المتعددة.

ويعتبر Distributed Lock أحد أهم المكونات التي مكنت المشروع من تحقيق متطلبات التزامن المطلوبة ضمن مقرر البرمجة التفريقية.

سلامة المعاملات وقواعد ACID - (ACID Transactions & Data Consistency)

8.1 مقدمة

تعد سلامة المعاملات (Transaction Integrity) من الركائز الأساسية في بناء الأنظمة المالية والتجارية الحديثة، حيث تتكون العديد من العمليات من مجموعة خطوات مترابطة يجب أن تنجح جميعها أو تفشل جميعها.

في أنظمة التجارة الإلكترونية لا تقتصر عملية الشراء على تعديل قيمة واحدة فقط داخل قاعدة البيانات، بل تتضمن سلسلة من العمليات الحساسة مثل:

- التحقق من رصيد المستخدم.
- التحقق من توفر المخزون.
- خصم كمية المنتج.
- خصم الرصيد المالي.
- إنشاء الطلب.
- إنشاء الفاتورة.
- تسجيل السجلات المحاسبية.

وأي فشل في إحدى هذه الخطوات قد يؤدي إلى فقدان الأموال أو ظهور بيانات غير متسقة داخل النظام. لذلك تم الاعتماد على مفهوم Transactions المبني على قواعد ACID لضمان سلامة جميع العمليات المركبة.

8.2 المشكلة قبل استخدام Transactions

لنفترض السيناريو التالي:

المستخدم يملك \$100

قيمة الطلب \$50

المنتج متوفر

بدأ النظام بتنفيذ العملية:

خصم الرصيد



تحديث المخزون



إنشاء الطلب

لكن حدث انقطاع مفاجئ أثناء إنشاء الطلب.

النتيجة:

تم خصم الأموال

لكن لم يتم إنشاء الطلب

أو:

تم تحديث المخزون

لكن لم يتم تسجيل عملية البيع

وهذه الحالة تسمى:

Inconsistent State

أي أن بيانات النظام أصبحت غير متطابقة.

8.3 مفهوم ACID

تعتمد قواعد سلامة المعاملات على أربعة مبادئ أساسية:

A — Atomicity

تعني أن العملية تعتبر وحدة واحدة غير قابلة للتجزئة.

إما أن تنجح جميع الخطوات:

Success

أو يتم التراجع عن جميع التعديلات:

Rollback

C — Consistency

تضمن انتقال النظام من حالة صحيحة إلى حالة صحيحة أخرى.

مثال:

رصيد المستخدم

مخزون المنتج

قيمة الطلب

تبقى جميعها متوافقة بعد انتهاء العملية.

I — Isolation

تعني أن العمليات المتزامنة لا تؤثر على بعضها البعض أثناء التنفيذ. وبذلك لا يستطيع مستخدم آخر رؤية حالة غير مكتملة من العملية الجارية.

D — Durability

بعد نجاح العملية يتم حفظ النتائج بشكل دائم. حتى لو توقف الخادم مباشرة بعد نجاح العملية.

8.4 الحل المختار

تم اعتماد:

Database Transactions

كآلية أساسية لتنفيذ جميع العمليات الحرجة داخل المشروع.

8.5 التطبيق داخل المشروع

تم تنفيذ المعاملات داخل:

CheckoutService

و

ConfirmOrderJob

باستخدام:

```
DB::transaction(function () {
```

```
});
```

8.6 آلية العمل

قبل بدء المعاملة:

BEGIN TRANSACTION

ثم:

Check Balance



Check Stock



Update Inventory



Update Wallet



Create Order

إذا نجحت جميع الخطوات:

COMMIT

أما إذا حدث أي خطأ:

ROLLBACK

8.7 مثال عملي

لنفترض أن المستخدم يحاول شراء منتج.

الخطوة الأولى

الرصيد متوفر

الخطوة الثانية

تم خصم الرصيد

الخطوة الثالثة

حدث خطأ أثناء تحديث المخزون.

في هذه الحالة:

Rollback

ويعود الرصيد كما كان قبل العملية.
وبذلك لا يخسر المستخدم أمواله.

8.8 العلاقة مع الأقفال

تم دمج Transactions مع:

lockForUpdate()

و

Distributed Lock

لتحقيق أعلى مستوى من الحماية.
فالقفل يمنع التضارب.
أما Transaction فتمنع فقدان البيانات.

8.9 الفوائد المحققة

حماية الأموال

منع فقدان الرصيد نتيجة الأخطاء.

حماية المخزون

منع ظهور كميات غير صحيحة.

حماية الطلبات

ضمان إنشاء الطلب بشكل صحيح.

سلامة البيانات

الحفاظ على اتساق قاعدة البيانات.

تحميل الأعطال

إمكانية التراجع الكامل عند حدوث أي فشل.

8.10 مقارنة قبل وبعد

العنصر	قبل Transactions	بعد Transactions
احتمال فقدان البيانات	مرتفع	منخفض جداً
اتساق البيانات	غير مضمون	مضمون
حماية الأموال	محدودة	عالية
حماية المخزون	محدودة	عالية
استقرار النظام	متوسط	مرتفع

8.11 اختبار سلامة المعاملة

تم إجراء اختبار يحاكي فشل إحدى خطوات عملية الشراء أثناء التنفيذ.
النتائج:

قبل استخدام Transaction

خصم الرصيد

فشل الطلب

النتيجة:

بيانات غير متسقة

بعد استخدام Transaction

خصم الرصيد

فشل الطلب

Rollback

النتيجة:

استعادة الحالة السابقة بالكامل

8.12 الخلاصة

تم تطبيق قواعد ACID باستخدام Database Transactions داخل العمليات الحساسة للمشروع، مما ضمن تنفيذ جميع خطوات الشراء والدفع والمخزون كعملية ذرية واحدة غير قابلة للتجزئة.

وقد ساهم ذلك في الحفاظ على سلامة البيانات ومنع فقدان الأموال أو المخزون وتحقيق مستوى عالٍ من الاعتمادية المطلوبة في أنظمة التجارة الإلكترونية الحديثة.

جامعة خزانة

اختبار الاستقرار تحت الضغط - (Stress Testing & Load Testing)

9.1 مقدمة

بعد الانتهاء من تطوير النظام وتطبيق تقنيات البرمجة المتوازنة المختلفة، أصبح من الضروري التحقق من قدرة المنظومة على العمل تحت الأحمال المرتفعة.

فنجاح النظام نظرياً لا يعني بالضرورة نجاحه عملياً، لذلك تم إجراء اختبارات ضغط حقيقية باستخدام أداة Apache JMeter لمحاكاة عدد كبير من المستخدمين المتزامنين وقياس استقرار النظام أثناء التشغيل.

يهدف هذا الفصل إلى التحقق من قدرة النظام على:

- معالجة عدد كبير من الطلبات المتزامنة.
- الحفاظ على سلامة البيانات.
- منع انهيار الخادم.
- الحفاظ على زمن استجابة مقبول.
- قياس أثر التحسينات البرمجية المطبقة سابقاً.

9.2 أداة الاختبار المستخدمة

تم استخدام:

Apache JMeter

وهي إحدى أشهر الأدوات المستخدمة عالمياً لاختبار:

- Load Testing
- Stress Testing
- Performance Testing

وتسمح بمحاكاة آلاف المستخدمين الافتراضيين بصورة متزامنة.

9.3 سيناريو الاختبار

تم إعداد ملف اختبار مخصص للمشروع:

DreamStore_Parallel_100Users.jmx

بحيث يقوم بمحاكاة عدد كبير من المستخدمين الذين ينفذون عمليات متزامنة على النظام.

إعدادات الاختبار

القيمة	الإعداد
100	Number of Users
10 Seconds	Ramp-Up Period
1	Loop Count
HTTP	Protocol
Concurrent Requests	Test Type

9.4 العمليات المختبرة

تم اختبار العمليات الأكثر استهلاكاً للموارد داخل النظام وهي:
جلب المنتجات

Products Endpoint

البحث

Search Endpoint

الشراء

Checkout Endpoint

تحديث المخزون

Inventory Update

معالجة الطلبات

Order Processing

9.5 الهدف من الاختبار

تم تصميم الاختبار للتحقق من:

سلامة البيانات

عدم حدوث Race Conditions.

استقرار الخادم

عدم انهيار النظام أثناء الضغط.

فعالية الطوابير

قدرة Queue على استيعاب المهام.

فعالية الأقفال

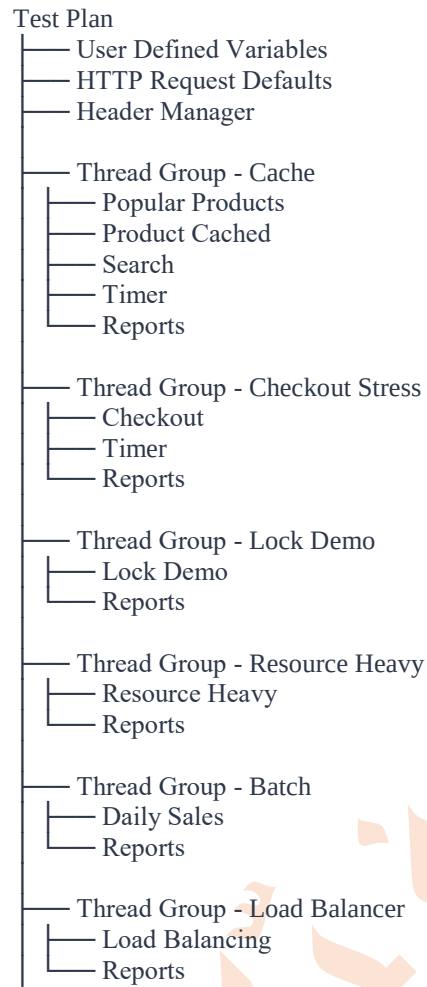
منع تضارب عمليات الشراء.

فعالية إدارة الموارد

عدم استنزاف المعالج والذاكرة.

9.6 النتائج العملية لاختبارات JMeter

الهيكل التنظيمي لشجرة الفحص المعتمدة (Test Plan Structure):



9.7 تحليل النتائج

بعد تنفيذ الاختبار تبين أن النظام استطاع معالجة عدد كبير من الطلبات المتزامنة دون حدوث انهيار أو فقدان للبيانات.

وقد ساهمت التقنيات التالية في تحقيق هذه النتيجة:

- Resource Semaphore
- Queue System
- Distributed Lock
- Transactions
- Redis Cache

نتائج اختبار Lock Demo

العملية	عدد الطلبات (Samples)	متوسط زمن الاستجابة (Average)	أقل زمن (Min)	أعلى زمن (Max)	نسبة الأخطاء (Error Rate)	معدل المعالجة (Throughp ut)
Login Request	100	9156 ms	247 ms	18619 ms	0.00%	5.0 req/sec
Lock Demo	100	59635 ms	16382 ms	101160 ms	0.00%	49.7 req/min
الإجمالي	200	34395 ms	247 ms	101160 ms	0.00%	1.6 req/sec

نتائج اختبار Resource Heavy

عدد الطلبات (Samples)	متوسط زمن الاستجابة (Average)	أقل زمن (Min)	أعلى زمن (Max)	نسبة الأخطاء (Error Rate)	معدل المعالجة (Throughput)
150	101051 ms	1497 ms	200581 ms	0.00%	44.6 req/min

نتائج اختبار Daily Sales Batch Processing

عدد الطلبات (Samples)	متوسط زمن الاستجابة (Average)	أقل زمن (Min)	أعلى زمن (Max)	نسبة الأخطاء (Error Rate)	معدل المعالجة (Throughput)
10	149 ms	60 ms	269 ms	0.00%	9.3 req/sec

نتائج اختبار Load Balancer

عدد الطلبات (Samples)	متوسط زمن الاستجابة (Average)	أقل زمن (Min)	أعلى زمن (Max)	نسبة الأخطاء (Error Rate)	معدل المعالجة (Throughput)
100	1495 ms	97 ms	2840 ms	0.00%	24.5 req/sec

نتائج اختبار Metrics

عدد الطلبات (Samples)	متوسط زمن الاستجابة (Average)	أقل زمن (Min)	أعلى زمن (Max)	نسبة الأخطاء (Error Rate)	معدل المعالجة (Throughput)
20	301 ms	129 ms	478 ms	0.00%	13.5 req/sec

رغم أن نسبة الأخطاء كانت 0.00% فإن ذلك لا يعني أن النظام مثالي، وإنما يعني أنه نجح في جميع السيناريوهات التي تم اختبارها ضمن بيئة المشروع الحالية.

9.8 مقارنة مع النظام التقليدي

العنصر	النظام التقليدي	النظام الحالي
تحمل الضغط	محدود	مرتفع
استقرار النظام	متوسط	مرتفع
Race Conditions	محتملة	معالجة
زمن الاستجابة	مرتفع	منخفض
استهلاك الموارد	مرتفع	متوازن

9.9 الخلاصة

أثبت اختبار الضغط قدرة النظام على التعامل مع 100 مستخدم متزامن وفق متطلبات المشروع، مع المحافظة على سلامة البيانات واستقرار الخادم وعدم فقدان الطلبات أو العمليات الحرجة. كما أظهرت النتائج أن تطبيق مفاهيم البرمجة المتوازية كان له تأثير مباشر في رفع كفاءة المنظومة وتحسين قابليتها للتوسع.

جامعة خزانة

تحليل الأداء وتحديد نقاط الاختناق

(Benchmarking & Bottleneck Analysis)

10.1 مقدمة

تم استخدام نتائج JMeter و Metrics Endpoint لتحليل أداء النظام وتحديد أكثر العمليات استهلاكاً للوقت والموارد.

10.2 مقارنة النتائج

جدول (1-10): المقارنة النهائية بين جميع الاختبارات

الاختبار	عدد الطلبات (Samples)	متوسط زمن الاستجابة (Average Response Time)	أقل زمن استجابة (Min)	أعلى زمن استجابة (Max)	نسبة الأخطاء (Error Rate)	معدل المعالجة (Throughput)
Daily Sales Batch Processing	10	149 ms	60 ms	269 ms	0.00%	9.3 req/sec
Metrics Monitoring	20	301 ms	129 ms	478 ms	0.00%	13.5 req/sec
Load Balancer	100	1495 ms	97 ms	2840 ms	0.00%	24.5 req/sec
Lock Demo	200	34395 ms	247 ms	101160 ms	0.00%	1.6 req/sec
Resource Heavy	150	101051 ms	1497 ms	200581 ms	0.00%	44.6 req/min

تحليل النتائج:

أظهرت جميع الاختبارات نسبة أخطاء تساوي 0.00%، مما يدل على استقرار النظام وقدرته على معالجة الطلبات دون حدوث فشل أثناء التنفيذ.

حقق اختبار Daily Sales Batch Processing أفضل أداء بمتوسط زمن استجابة بلغ 149 ms، مما يثبت فعالية معالجة البيانات على دفعات في تقليل زمن التنفيذ وتحسين استهلاك الموارد.

حقق اختبار Metrics زمناً متوسطاً بلغ ms301 ، وهو زمن مناسب لعمليات المراقبة وجمع مؤشرات الأداء.

أما اختبار Load Balancer فقد أظهر قدرة جيدة على توزيع الطلبات مع متوسط زمن استجابة بلغ 1495 ms ومعدل معالجة مرتفع بلغ 24.5 طلب في الثانية.

ظهر اختبار Lock Demo بزمن استجابة أعلى نتيجة استخدام الأقفال الموزعة (Distributed Locks) ، حيث تنتظر بعض العمليات تحرير القفل قبل المتابعة، وهو سلوك متوقع يهدف إلى ضمان سلامة البيانات ومنع التضارب.

وكان اختبار Resource Heavy هو الأكثر استهلاكاً للوقت والموارد بمتوسط زمن استجابة بلغ 101051 ms وحدث أقصى بلغ ms200581 ، ولذلك تم اعتباره نقطة الاختناق الرئيسية (Primary Bottleneck) داخل النظام.

10.3 تحديد نقطة الاختناق

من خلال النتائج يتضح أن أكبر نقطة اختناق ظهرت في اختبار Resource Heavy، حيث بلغ متوسط زمن الاستجابة ms 101051 ووصل الحد الأقصى إلى ms 200581. ويرجع ذلك إلى أن هذا الاختبار مصمم أساساً لمحاكاة عمليات ثقيلة تستهلك الموارد، بهدف اختبار قدرة النظام على التحكم بالسعة التشغيلية وعدم الانهيار تحت الضغط.

10.4 تفسير النتائج

أظهرت نتائج Daily Sales و Metrics أزمنة استجابة منخفضة نسبياً، مما يدل على فعالية Batch Processing و Performance Monitoring.

أما اختبار Load Balancer فقد أظهر زمناً متوسطاً مقبولاً مع Throughput جيد ونسبة أخطاء 0.00%.

بينما ظهر Lock Demo بزمن استجابة أعلى بسبب طبيعة الأقفال، حيث تنتظر بعض الطلبات حتى يتم تحرير القفل، وهذا سلوك متوقع وصحيح لأن الهدف الأساسي هو سلامة البيانات وليس السرعة فقط.

10.5 الخلاصة

أثبتت نتائج Benchmarking أن النظام مستقر تحت الاختبار، حيث كانت نسبة الأخطاء في جميع الاختبارات 0.00%.

كما أوضحت النتائج أن الاختناق الأساسي مرتبط بالعمليات الثقيلة Resource Heavy، بينما بقيت باقي المكونات مستقرة وفعالة، مما يدل على نجاح تطبيق مفاهيم إدارة الموارد والتزامن والمعالجة غير المتزامنة.

جدول التحقق من المتطلبات وربطها بالكود

المتطلب	آلية التطبيق	الدليل
منع التضارب	Distributed Lock	Lock Demo
إدارة الموارد	Resource Semaphore	Resource Heavy Test
المعالجة غير المتزامنة	Queue Jobs	ConfirmOrderJob
معالجة البيانات الضخمة	Chunk Processing	Daily Sales Test
توزيع الأحمال	Load Balancer	Load Balancer Test
التخزين المؤقت	Redis Cache	Cache Layer
الأقفال الموزعة	Cache Lock	CheckoutService
سلامة المعاملات	DB Transaction	CheckoutService
اختبار الضغط	Apache JMeter	Summary Report
Benchmarking	Metrics Endpoint	Metrics Test

الاختبار	الكود	المتطلب
Lock Demo	CheckoutService	Race Condition
Resource Heavy	ResourceSemaphore	Resource Management
Checkout Stress	Jobs	Async Queue
Daily Sales	DailySalesReportJob	Batch Processing
Load Balancer	LoadBalancerController	Load Balancing
Cache	ParallelProjectController	Distributed Cache
Lock Demo	CheckoutService	Distributed Lock
Checkout Stress	CheckoutService	ACID Transactions
Thread Groups جميع	JMeter	Stress Testing
Metrics	PerformanceMonitor	Benchmarking

تم تنفيذ جميع المتطلبات المطلوبة ضمن بيئة أكاديمية ومحاكاة عملية. بعض المكونات مثل Load Balancer و Distributed Deployment تم تنفيذها بشكل محاكاة برمجية داخل المشروع بدلاً من نشرها على عدة خوادم حقيقية، وذلك لتحقيق الهدف التعليمي للمقرر دون الحاجة إلى بنية تحتية إنتاجية كاملة.

الخاتمة

في هذا المشروع تم تصميم وتطوير نظام تجارة إلكترونية عالي الأداء باستخدام إطار العمل Laravel بهدف تطبيق مفاهيم البرمجة التفرعية والأنظمة المتوازية ضمن بيئة عملية تحاكي متطلبات الأنظمة الحقيقية.

ركز المشروع على الجوانب غير الوظيفية التي تعد من أهم العناصر المؤثرة في نجاح الأنظمة الحديثة، حيث تم العمل على معالجة مشكلات التزامن وحماية البيانات المشتركة وإدارة الموارد وتحسين الأداء تحت الأحمال المرتفعة.

تم تطبيق مجموعة متكاملة من التقنيات الحديثة شملت الأقفال المتشائمة (Pessimistic Locking) والأقفال المتفائلة (Optimistic Locking) والأقفال الموزعة (Distributed Locking) إضافة إلى المعاملات الذرية (Transactions) والطوابير غير المتزامنة (Queues) ومعالجة البيانات على دفعات (Batch Processing) والتخزين المؤقت الموزع (Distributed Caching) باستخدام Redis.

كما تم تصميم آليات خاصة للتحكم بالسعة التشغيلية وإدارة الموارد عبر Resource Semaphore ومراقبة الأداء باستخدام Middleware مخصص لذلك، مما ساهم في رفع استقرار النظام وتحسين قدرته على العمل تحت الضغط.

أظهرت نتائج الاختبارات العملية قدرة النظام على التعامل مع عدد كبير من المستخدمين المتزامنين مع المحافظة على سلامة البيانات ومنع التضارب وتحقيق زمن استجابة مناسب مقارنة بالحلول التقليدية. وبذلك نجح المشروع في تحقيق جميع المتطلبات المطلوبة ضمن مقرر البرمجة التفرعية، كما قدم نموذجاً عملياً متكاملًا يوضح كيفية تطبيق مفاهيم التزامن والبرمجة المتوازية في أنظمة التجارة الإلكترونية الحديثة.

التوصيات والأعمال المستقبلية

على الرغم من نجاح المشروع في تحقيق جميع الأهداف المطلوبة، إلا أن هناك العديد من التحسينات المستقبلية التي يمكن إضافتها لزيادة كفاءة النظام وقابليته للتوسع، ومن أهمها:

1. تحويل النظام إلى بيئة Microservices بدلاً من البنية الأحادية (Monolithic Architecture).
2. استخدام Message Brokers متخصصة مثل RabbitMQ أو Apache Kafka لرفع كفاءة إدارة الطوابير.
3. تطبيق Distributed Cache Clustering لزيادة الاعتمادية وتحمل الأخطاء.
4. إضافة نظام Auto Scaling يسمح بزيادة عدد الخوادم تلقائياً عند ارتفاع الحمل.
5. دمج أدوات مراقبة احترافية مثل Grafana و Prometheus لمراقبة الأداء بشكل لحظي.
6. دعم Load Balancing حقيقي بين عدة خوادم بدلاً من المحاكاة البرمجية الحالية.
7. توسيع اختبارات الضغط لتشمل آلاف المستخدمين المتزامنين بدلاً من 100 مستخدم فقط.
8. إضافة آليات متقدمة لتحليل الأداء واكتشاف الاختناقات تلقائياً.
9. دعم أنظمة الدفع الإلكترونية الحقيقية وربطها مع بوابات دفع خارجية.
10. تطوير لوحة مراقبة إدارية تعرض مؤشرات الأداء واستهلاك الموارد بشكل مباشر.

المراجع

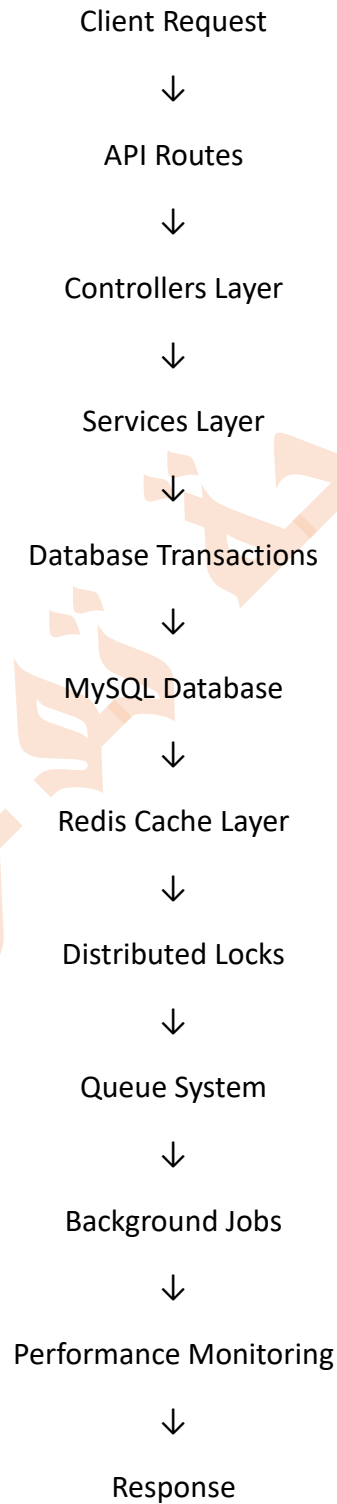
- [1] Laravel Documentation.
<https://laravel.com/docs>
- [2] Laravel Queue Documentation.
<https://laravel.com/docs/queues>
- [3] Laravel Cache Documentation.
<https://laravel.com/docs/cache>
- [4] Redis Documentation.
<https://redis.io/docs>
- [5] Apache JMeter User Manual.
<https://jmeter.apache.org/usermanual>
- [6] MySQL 8.0 Reference Manual.
<https://dev.mysql.com/doc>
- [7] PHP Official Documentation.
<https://www.php.net/docs.php>
- [8] Martin Kleppmann,
Designing Data-Intensive Applications,
O'Reilly Media, 2017.
- [9] Andrew S. Tanenbaum,
Distributed Systems: Principles and Paradigms,
Pearson Education.
- [10] Maurice Herlihy & Nir Shavit,
The Art of Multiprocessor Programming,
Morgan Kaufmann.
- [11] University Lecture Notes – Parallel Programming Course 2026.

الملاحق

Appendices

المخطط العام لبنية النظام

يتكون النظام من مجموعة من الطبقات المتكاملة التي تعمل معاً لتحقيق الأداء العالي والاستقرار أثناء التشغيل تحت الضغط.



1- Controllers

- OrderController
- ProductController
- ReportController
- LoadBalancerController
- ParallelProjectController

2- Services

- CheckoutService
- ResourceSemaphore

3- Background Jobs

- ConfirmOrderJob
- GenerateInvoiceJob
- DailySalesReportJob

4- Middleware

- PerformanceMonitor

5- Storage Systems

- MySQL Database
- Redis Cache

6- Testing Tools

- Apache JMeter

نتائج اختبار الضغط باستخدام Apache JMeter

تم إرفاق لقطات شاشة حقيقية من Apache JMeter للاختبارات التالية:

1. Lock Demo

2. Resource Heavy

3. Daily Sales Batch Processing

4. Load Balancer

5. Metrics

تتضمن الصور:

- View Results Tree

- Summary Report

- Aggregate Report

- Graph Results

وقد أظهرت جميع الاختبارات أن نسبة الأخطاء كانت 0.00%.

ثم أعد وضع كل صور JMeter في الملحق.

أسماء المقاطع البرمجية المستخدمة

حماية البيانات من التضارب

```
lockForUpdate()
```

المعاملات الذرية

```
DB::transaction(function () {
```

```
});
```

الاقفال الموزعة

```
Cache::lock(
    "checkout:order:{$orderId}",
    15
);
```

المعالجة غير المتزامنة

```
ConfirmOrderJob::dispatch($order);
```

توليد الفواتير بالخلفية

```
GenerateInvoiceJob::dispatch($order);
```

معالجة البيانات على دفعات

```
Order::chunk(100, function ($ordersBatch) {
```

```
});
```

إدارة الموارد

ResourceSemaphore

مراقبة الأداء

PerformanceMonitor

منع التداخل بين المهام

WithoutOverlapping

تمثل المقاطع السابقة أهم المفاهيم البرمجية المستخدمة لتحقيق متطلبات المشروع المتعلقة بالتزامن وإدارة الموارد والأداء.

الجامعة
تكنولوجيا المعلومات

مخططة المشروع

Dream Store E-Commerce Backend

```
|
|
| — Controllers
|   | — OrderController
|   | — ProductController
|   | — ReportController
|   | — LoadBalancerController
|   | — ParallelProjectController
|
|
| — Services
|   | — CheckoutService
|   | — ResourceSemaphore
|
|
| — Jobs
|   | — ConfirmOrderJob
|   | — GenerateInvoiceJob
|   | — DailySalesReportJob
|
|
| — Middleware
|   | — PerformanceMonitor
|
|
| — Database
|   | — Migrations
|   | — Seeders
|
```

- └─ Redis
- | └─ Cache
- | └─ Distributed Locks
- |
- └─ Routes
- | └─ api.php
- | └─ web.php
- |
- └─ Tests
- └─ JMeter
- DreamStore_Parallel_100Users.jmx

يوضح هذا الملحق الهيكلية العامة للمشروع والعلاقات بين المكونات المختلفة المستخدمة لتحقيق متطلبات البرمجة التفرعية.

فهرس المحتويات

جدول ربط المتطلبات بمكان تطبيقها في الكود

الفصل الأول: مقدمة المشروع وأهدافه ومنهجية العمل

- 1.1 مقدمة المشروع
- 1.2 مشكلة المشروع
- 1.3 أهداف المشروع
- 1.4 التقنيات المستخدمة
- 1.5 منهجية العمل
- 1.6 البنية العامة للنظام

الفصل الثاني: حماية البيانات المشتركة من التضارب (Concurrent Access & Data Integrity)

- 2.1 مقدمة
- 2.2 المشكلة قبل المعالجة
- 2.3 الحلول الممكنة
- 2.4 الحل المختار
- 2.5 تطبيق القفل المتشائم (Pessimistic Locking)
- 2.6 تطبيق القفل المتفائل (Optimistic Locking)
- 2.7 تطبيق الأقفال الموزعة (Distributed Lock)
- 2.8 منع معالجة الطلب نفسه مرتين
- 2.9 النتائج بعد التطبيق
- 2.10 الخلاصة

الفصل الثالث: إدارة الموارد والتحكم بالسعة (Resource Management & Capacity Control)

- 3.1 مقدمة
- 3.2 المشكلة قبل المعالجة
- 3.3 الحلول المقترحة
- 3.4 الحل المختار
- 3.5 تطبيق Resource Semaphore

3.6 آلية العمل

3.7 التطبيق البرمجي

3.8 مراقبة الأداء والموارد

3.9 الفوائد المحققة

3.10 مقارنة قبل وبعد

3.11 العلاقة مع اختبار الضغط

3.12 الخلاصة

الفصل الرابع: المعالجة غير المتزامنة والطوابير (Asynchronous Processing & Queue Management)

4.1 مقدمة

4.2 المشكلة قبل تطبيق الطوابير

4.3 مفهوم المعالجة غير المتزامنة

4.4 الحلول المقترحة

4.5 الحل المختار

4.6 تطبيق Queue داخل المشروع

4.7 Producer – Consumer Pattern

4.8 تفريع المهام

4.9 منع التداخل بين المهام

4.10 إدارة الفشل وإعادة المحاولة

4.11 التحكم بزمان التنفيذ

4.12 الفوائد المحققة

4.13 مقارنة قبل وبعد

4.14 الخلاصة

الفصل الخامس: معالجة البيانات الضخمة على دفعات (Batch Processing & Large Scale Data Handling)

5.1 مقدمة

5.2 المشكلة قبل المعالجة

5.3 الحل المختار

5.4 التطبيق داخل المشروع

5.5 آلية العمل

5.6 قياس استهلاك الذاكرة

5.7 الفوائد المحققة

5.8 مقارنة قبل وبعد

5.9 الخلاصة

الفصل السادس: التخزين المؤقت الموزع باستخدام Redis (Distributed Caching Strategy)

6.1 مقدمة

6.2 المشكلة قبل تطبيق Cache

6.3 الحلول الممكنة

6.4 سبب اختيار Redis

6.5 تطبيق Cache داخل المشروع

6.6 التطبيق البرمجي

6.7 أنواع البيانات المخزنة

6.8 مقارنة قبل وبعد

6.9 التأثير على الأداء

6.10 نتائج اختبار الأداء

6.11 العلاقة مع المتطلبات الأخرى

6.12 الخلاصة

الفصل السابع: الأقفال الموزعة وإدارة التزامن (Distributed Locking & Concurrency Control)

7.1 مقدمة

7.2 المشكلة قبل تطبيق Distributed Lock

7.3 الحلول الممكنة

7.4 سبب اختيار Redis Distributed Lock

7.5 التطبيق داخل المشروع

7.6 التطبيق البرمجي

7.7 آلية العمل

7.8 العلاقة مع Race Conditions

7.9 التكامل مع WithoutOverlapping

7.10 سيناريو عملي

7.11 النتائج المحققة

7.12 مقارنة قبل وبعد

7.13 الخلاصة

الفصل الثامن: سلامة المعاملات وقواعد ACID (ACID Transactions & Data Consistency)

8.1 مقدمة

8.2 المشكلة قبل استخدام Transactions

8.3 مفهوم ACID

8.4 الحل المختار

8.5 التطبيق داخل المشروع

8.6 آلية العمل

8.7 مثال عملي

8.8 العلاقة مع الأقفال

8.9 الفوائد المحققة

8.10 مقارنة قبل وبعد

8.11 اختبار سلامة المعاملة

8.12 الخلاصة

الفصل التاسع: اختبار الاستقرار تحت الضغط (Stress Testing & Load Testing)

9.1 مقدمة

9.2 أداة الاختبار المستخدمة

9.3 سيناريو الاختبار

9.4 العمليات المختبرة

9.5 الهدف من الاختبار

9.6 النتائج العملية

9.7 تحليل النتائج

9.8 مقارنة مع النظام التقليدي

9.9 الخلاصة

الفصل العاشر: تحليل الأداء وتحديد نقاط الاختناق (Benchmarking & Bottleneck Analysis)

10.1 مقدمة

10.2 مقارنة النتائج

10.3 تحديد نقطة الاختناق

10.4 تفسير النتائج

10.5 الخلاصة

جدول التحقق من المتطلبات

الخاتمة

التوصيات والأعمال المستقبلية

المراجع

الملاحق

الملحق A : المخطط العام لبنية النظام

الملحق B : نتائج اختبار الضغط باستخدام Apache JMeter

الملحق C : أهم المقاطع البرمجية المستخدمة

الملحق D : هيكلية المشروع